Danish Kings. Fix variable names and check results.

Søren L.F. Lageri

2026-03-06

Here we load the `tidyverse` library and its packages.

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.0     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.2     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.1
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

The task here is to load your Danish Monarchs csv into R using the `tidyverse` toolkit, calculate and explore the kings' duration of reign with pipes `%>%` in `dplyr` and plot it over time.

Load the kings

Make sure to first create an `.Rproj` workspace with a `data/` folder where you place either your own dataset or the provided `kings.csv` dataset.

1. Look at the dataset that are you loading and check what its columns are separated by? (hint: open it in plain text editor to see) List what is the

separator/delimiter: ; (semi colon)

2. Create a `kings` object in R with the different functions below and inspect the different outputs.

- `read.csv()`
- `read_csv()`
- `read.csv2()`
- `read_csv2()`

```
# FILL IN THE CODE BELOW and review the outputs.

kings1 <- read.csv("C:/Users/sl200/OneDrive - Aarhus universitet/6. semester/Digitale arkiver og metode

kings2 <- read_csv("C:/Users/sl200/OneDrive - Aarhus universitet/6. semester/Digitale arkiver og metode
```

```
## Rows: 57 Columns: 1
## -- Column specification --------------------------------------------------------
## Delimiter: ","
```

```
## chr (1): Name;Birth_year;Born_certain;Born_approx;Born_unknown;Dead_year;Dea...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
kings3 <- read.csv2("C:/Users/sl200/OneDrive - Aarhus universitet/6. semester/Digitale arkiver og metod

kings4 <- read_csv2("C:/Users/sl200/OneDrive - Aarhus universitet/6. semester/Digitale arkiver og metod
```

```
## i Using "',' as decimal and "'.'" as grouping mark. Use `read_delim()` for more control.
## Rows: 57 Columns: 11-- Column specification --------------------------------------------------------
## Delimiter: ";"
## chr (1): Name
## dbl (4): Birth_year, Dead_year, Era_start, Era_finished
## lgl (6): Born_certain, Born_approx, Born_unknown, Dead_certain, Dead_approx,...
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# We cannot only use "data/U11/Monark_ark_Danish.csv" to load the file.
# This is due to the R Markdown file being stored in a sub-directory in the R project file.
# Therefore, we load the file by providing the path to the file directly.
```

Answer:

1. Which of these functions is a `tidyverse` function? Read data with it below into a `kings` object. When we inspect the `read.csv` and `read_csv` functions we find which library they are from.

```
find("read.csv")
```

```
## [1] "package:utils"
```

```
find("read_csv")
```

```
## [1] "package:readr"
```

From the start of the document, where we loaded the `tidyverse` library, we can see that the package `readr` is a part of `tidyverse`. Therefore, `read_csv` and `read_csv2` are `tidyverse` functions. Meanwhile, the `util` package is a part of the base R packages.

2. What is the result of running `class()` on the `kings` object created with a tidyverse function. Using `class()` on the functions `read.csv` and `read.csv2` it returns a data.frame

Using `class()` on the tidyverse functions `read_csv` and `read_csv2` it returns multiple classes these being:

- spec_tbl_df: "specification table data frame" and is a specific type of tibble created by `readr` package from the `tidyverse`
- tbl_df: Also a tibble created by `tidyverse`.
- tbl: A parent class for tibbles.
- data.frame: Base R data frame class.

3. How many columns does the object have when created with these different functions? Objects created with `.csv` and `_csv` all have the variables within the first column since it cannot separate. But by using `.csv2` and `_csv2` the variables are separated to be within their own respective column, since R interprets semi colon (;) to be the column separator.

Generally, when working with the column, the columns remain the same but a tibble remains a tibble when you extract a single column from it, whereas a data frame becomes a vector when you select a single column from it. Therefore, for our case we will stay with tibbles.

```r
# COMPLETE THE BLANKS BELOW WITH YOUR CODE, then turn the 'eval' flag in this chunk to TRUE.

kings_list <- list( # Here we define a list for the four loaded datasets.
  kings1 = kings1,
  kings2 = kings2,
  kings3 = kings3,
  kings4 = kings4
)

# We create a for-loop that iterates through the names of the datasets in the list.
# The function names(kings_list) returns the labels "kings1", "kings2", "kings3", and "kings4".

# Inside the loop, we extract the corresponding dataset from the list with and the element within with

#cat(), which concatenates text and prints it. (Works like making vectors with c(X,X,X,X) except it als
# \n creates a new line, so the text is not in one line (line break).

for (name in names(kings_list)) {

  data <- kings_list[[name]]

  cat("\nDataset:", name, "\n",
      "Data type:", class(data), "\n",
      "Columns:", ncol(data), "\n",
      "Rows:", nrow(data), "\n",
      "Size:", size_sum(data), "\n",
      "Column names:", colnames(data), "\n")
}
```

4. Show the dataset so that we can see how R interprets each column

```
##
## Dataset: kings1
##  Data type: data.frame
##  Columns: 1
##  Rows: 57
##  Size: [57 x 1]
##  Column names: Name.Birth_year.Born_certain.Born_approx.Born_unknown.Dead_year.Dead_certain.Dead_appr
##
## Dataset: kings2
##  Data type: spec_tbl_df tbl_df tbl data.frame
##  Columns: 1
##  Rows: 57
##  Size: [57 x 1]
##  Column names: Name;Birth_year;Born_certain;Born_approx;Born_unknown;Dead_year;Dead_certain;Dead_appr
##
## Dataset: kings3
##  Data type: data.frame
##  Columns: 11
##  Rows: 57
##  Size: [57 x 11]
##  Column names: Name Birth_year Born_certain Born_approx Born_unknown Dead_year Dead_certain Dead_appr
##
```

```
## Dataset: kings4
##  Data type: spec_tbl_df tbl_df tbl data.frame
##  Columns: 11
##  Rows: 57
##  Size: [57 x 11]
##  Column names: Name Birth_year Born_certain Born_approx Born_unknown Dead_year Dead_certain Dead_app
```

I will stick with kings4, which utilities `read_csv2` function from `tidyverse`, since our dataset has semi colon (;) as the separator.

```
head(kings4)
```

```
## # A tibble: 6 x 11
##   Name   Birth_year Born_certain Born_approx Born_unknown Dead_year Dead_certain
##   <chr>       <dbl> <lgl>        <lgl>       <lgl>            <dbl> <lgl>
## 1 Gorm ~        913 FALSE        FALSE       TRUE               958 FALSE
## 2 Haral~         NA FALSE        FALSE       TRUE               987 FALSE
## 3 Svend~         NA FALSE        FALSE       TRUE              1014 FALSE
## 4 Haral~         NA FALSE        FALSE       TRUE              1018 FALSE
## 5 Knud ~        995 FALSE        TRUE        FALSE             1035 FALSE
## 6 Harde~       1020 FALSE        TRUE        FALSE             1042 FALSE
## # i 4 more variables: Dead_approx <lgl>, Dead_unknown <lgl>, Era_start <dbl>,
## #   Era_finished <dbl>
```

Calculate the duration of reign for all the kings in your table

You can calculate the duration of reign in years with `mutate` function by subtracting the equivalents of your `startReign` from `endReign` columns and writing the result to a new column called `duration`. But first you need to check a few things:

- Is your data messy? Fix it before re-importing to R
- Do your start and end of reign columns contain NAs? Choose the right strategy to deal with them: `na.omit()`, `na.rm=TRUE`, `!is.na()`

Create a new column called `duration` in the kings dataset, utilizing the `mutate()` function from tidyverse. Check with your group to brainstorm the options.

```
# YOUR CODE

# Column 10 is Era_start and column 11 is Era_finished.

# We want to define a new column that subtracts the end of the era with the start.
# We want to keep rows where Era_start has no NA OR Era_finished has no NA.
# This results in rows where both are NA get removed.

Reign_duration <- kings4 %>% # A pipe (%>%) is used to combine multiple functions.
  filter(!is.na(Era_start) | !is.na(Era_finished)) %>% # Here we filter out NA. (not necessary for us)
  mutate(Reign_duration = Era_finished - Era_start) # Here we create a new column.

# The duration column is placed at 12 and has 57 values.
Reign_duration[12]
```

```
## # A tibble: 57 x 1
##    Reign_duration
##             <dbl>
##  1             22
##  2             29
```

```
##  3             14
##  4              4
##  5             17
##  6              7
##  7              5
##  8             27
##  9              6
## 10              6
## # i 47 more rows
```

```
#Reign_duration["Reign_duration"] # Can also be used but Works best for one column.
```

Calculate the average duration of reign for all rulers

Do you remember how to calculate an average on a vector object? If not, review the last two lessons and remember that a column is basically a vector. So you need to subset your `kings` dataset to the `duration` column. If you subset it as a vector you can calculate average on it with `mean()` base-R function. If you subset it as a tibble, you can calculate average on it with `summarize()` tidyverse function. Try both ways!

- You first need to know how to select the relevant `duration` column. What are your options?
- Is your selected `duration` column a tibble or a vector? The `mean()` function can only be run on a vector. The `summarize()` function works on a tibble.
- Are you getting an error that there are characters in your column? Coerce your data to numbers with `as.numeric()`.
- Remember to handle NAs: `mean(X, na.rm=TRUE)`

```
# YOUR CODE

Average_reign_duration_baseR <-
  mean(as.numeric(Reign_duration$Reign_duration),na.rm = TRUE)

cat("Base-R mean:",round(Average_reign_duration_baseR,digits=1),"\n") # We round it to two digits.
```

```
## Base-R mean: 19.3
```

```
 # cat functions do not work with pipes, so we type it separately
```

```
# We take the mean of the duration of the reigns and remove NA.

Average_reign_duration <- Reign_duration %>%
  summarize(avg_duration = mean(Reign_duration, na.rm = TRUE))

Average_reign_duration[1,1] # The new tibble has one element at [1,1]
```

```
## # A tibble: 1 x 1
##   avg_duration
##          <dbl>
## 1         19.3
```

The average duration of the rulers was 19.3 years.

How many and which kings enjoyed a longer-than-average duration of reign?

You have calculated the average duration above. Use it now to `filter()` the `duration` column in `kings` dataset. Display the result and also count the resulting rows with `count()`

```
# YOUR CODE
```

```r
longer_than_avg_reigns <- Reign_duration %>% # We use the pipe to push it
  filter(Reign_duration > Average_reign_duration$avg_duration) %>%
  select(Name,Era_start,Era_finished,Reign_duration)
# $ extracts the avg_duration column from the Average_rule_duration tibble.

longer_than_avg_reigns
```

```
## # A tibble: 26 x 4
##    Name                   Era_start Era_finished Reign_duration
##    <chr>                      <dbl>        <dbl>          <dbl>
##  1 Gorm den Gamle               936          958             22
##  2 Harald 1. Blåtand            958          987             29
##  3 Svend 2. Estridsen          1047         1074             27
##  4 Niels                       1104         1134             30
##  5 Valdemar 1. den Store       1157         1182             25
##  6 Knud 4.                     1182         1202             20
##  7 Valdemar 2. Sejr            1202         1241             39
##  8 Erik 5. Klipping            1259         1286             27
##  9 Erik 6. Menved              1286         1319             33
## 10 Valdemar 4. Atterdag        1340         1375             35
## # i 16 more rows
```

```r
count(longer_than_avg_reigns)
```

```
## # A tibble: 1 x 1
##       n
##   <int>
## 1    26
```

There are 26 kings who had a longer-than-average duration of reign.

How many days did the three longest-ruling monarchs rule?

- Sort kings by reign `duration` in the descending order. Select the three longest-ruling monarchs with the `slice()` function
- Use `mutate()` to create `Days` column where you calculate the total number of days they ruled
- BONUS: consider the transition year (with 366 days) in your calculation!

```r
# YOUR CODE

sorted_kings <- Reign_duration %>% # First we sort the reign duration.
  arrange(desc(Reign_duration)) # We arrange it in descending order with biggest to smallest.

three_longest_reigns <- sorted_kings %>%  # Now having them sorted we want the first three.
  slice(1:3) # Here we select the first three rows from the sorted_kings.

years_to_days_leap <- 365.25 # 365 days a year and one extra year every fourth.

three_longest_reigns_days <- three_longest_reigns %>%
  mutate(Reign_days = Reign_duration*years_to_days_leap) # We create a new column "Reign_days".

three_column_focused <- three_longest_reigns_days %>%
  select(Name, Reign_duration, Reign_days) # We select column 1, 12 and 13.

three_column_focused
```

```
## # A tibble: 3 x 3
##   Name               Reign_duration Reign_days
##   <chr>                       <dbl>      <dbl>
## 1 Christian 4.                   60     21915
## 2 Margrethe 2.                   52     18993
## 3 Erik 7. af Pommern             43     15706.
```

```
# The code from before can be optimised to one continuous pipe.

three_longest_reigns_days_optimised <- Reign_duration %>%
  arrange(desc(Reign_duration)) %>% # Arranges so biggest to smallest.
  slice(1:3) %>% # Selects the first three rows.
  mutate(Reign_days = Reign_duration*365.25) %>%  # Converts to days (with leap) and new column.
  select(Name, Reign_duration, Reign_days) # We select column 1, 12 and 13.

three_longest_reigns_days_optimised
```

```
## # A tibble: 3 x 3
##   Name               Reign_duration Reign_days
##   <chr>                       <dbl>      <dbl>
## 1 Christian 4.                   60     21915
## 2 Margrethe 2.                   52     18993
## 3 Erik 7. af Pommern             43     15706.
```

The three longest-ruling monarchs and their rules were:

Challenge: Plot the kings' duration of reign through time

What is the long-term trend in the duration of reign among Danish monarchs? How does it relate to the historical violence trends ?

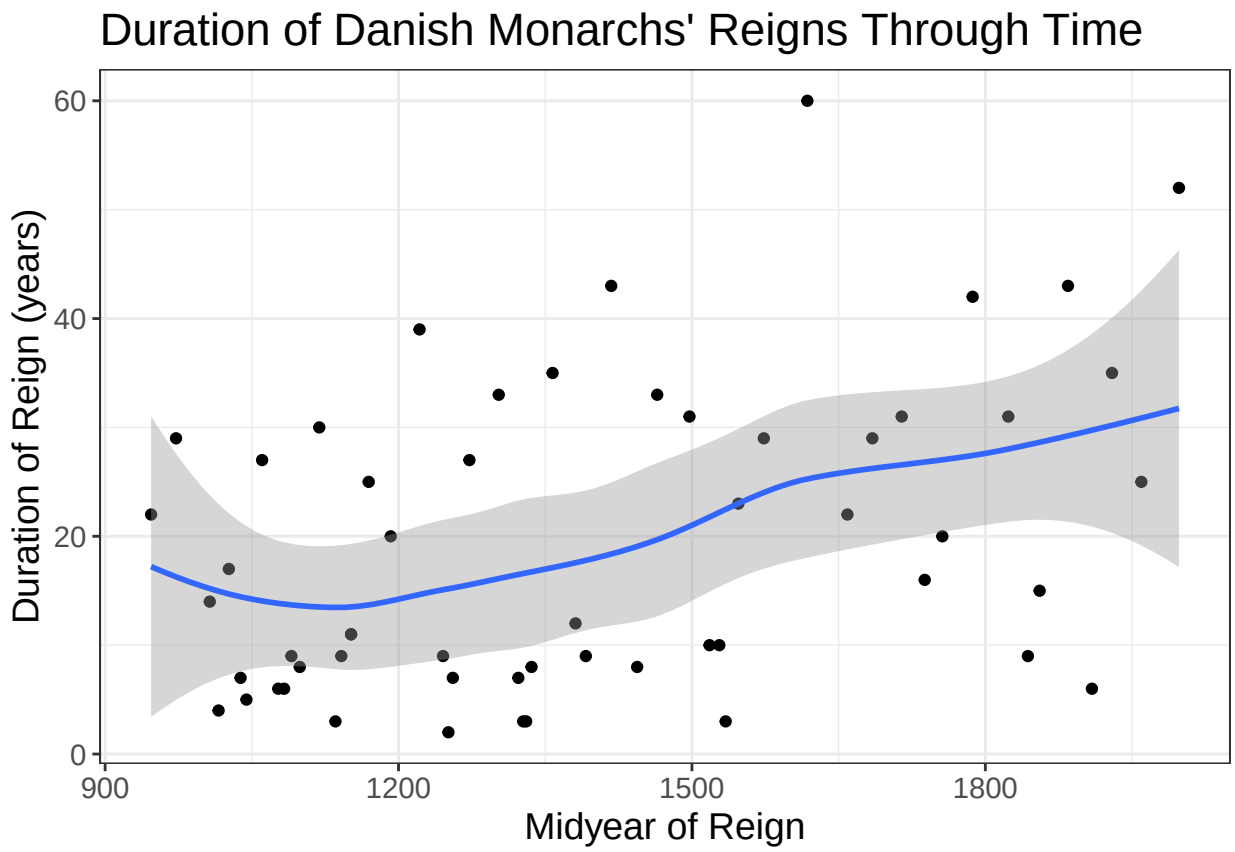
- Try to plot the duration of reign column in `ggplot` with `geom_point()` and `geom_smooth()`
- In order to peg the duration (which is between 1-99) somewhere to the x axis with individual centuries, I recommend creating a new column `midyear` by adding to `startYear` the product of `endYear` minus the `startYear` divided by two (`startYear + (endYear-startYear)/2`).
- Now you can plot the kings dataset, plotting `midyear` along the x axis and `duration` along y axis
- BONUS: add a title, nice axis labels to the plot and make the theme B&W and font bigger to make it nice and legible!

```
# YOUR CODE

midyear_Reign <- Reign_duration %>%
  mutate(midyear = Era_start + (Era_finished-Era_start)/2)

#midyear_definition[10:13]

ggplot(midyear_Reign, aes(x=midyear,y=Reign_duration)) +
  geom_point() + # gives data points in the plot
  geom_smooth() + # Fits a curve to the points.
  labs(
    title = "Duration of Danish Monarchs' Reigns Through Time",
    x = "Midyear of Reign",
    y = "Duration of Reign (years)"
  ) +
  theme_bw() +
  theme(text = element_text(size = 14))
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

```
## Warning: Removed 1 row containing non-finite outside the scale range
## (`stat_smooth()`).
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).
```



And to submit this rmarkdown, knit it into html. But first, clean up the code chunks, adjust the date, rename the author and change the `eval=FALSE` flag to `eval=TRUE` so your script actually generates an output. Well done!